

Chapitre 3

Utilisation de Hadoop

Manipulation de HDFS
Développement d'une application MapReduce



Plan

- Introduction
- Cloudera CDH
- Manipulation de HDFS
- Programmation Hadoop : Jobs MapReduce

Introduction

- Pour utiliser **Hadoop** deux solutions sont disponibles.
 - La première est d'utiliser **la version proposée par la fondation Apache**. Cette version est celle de référence et contient le noyau et quelques interfaces d'administration très simplifiée.
 - La seconde solution est d'utiliser **les distributions fournies par des entreprises qui font du service autour de Hadoop**. Comme il y a une valeur ajoutée non négligeable, ces distributions fournissent des services payants. Sans être exhaustif, voici trois sociétés qui font de Hadoop leur spécialité :
 - Cloudera : <http://www.cloudera.com/>
 - Hortonworks : <http://hortonworks.com/>
 - MapR Technologies : <http://www.mapr.com/>

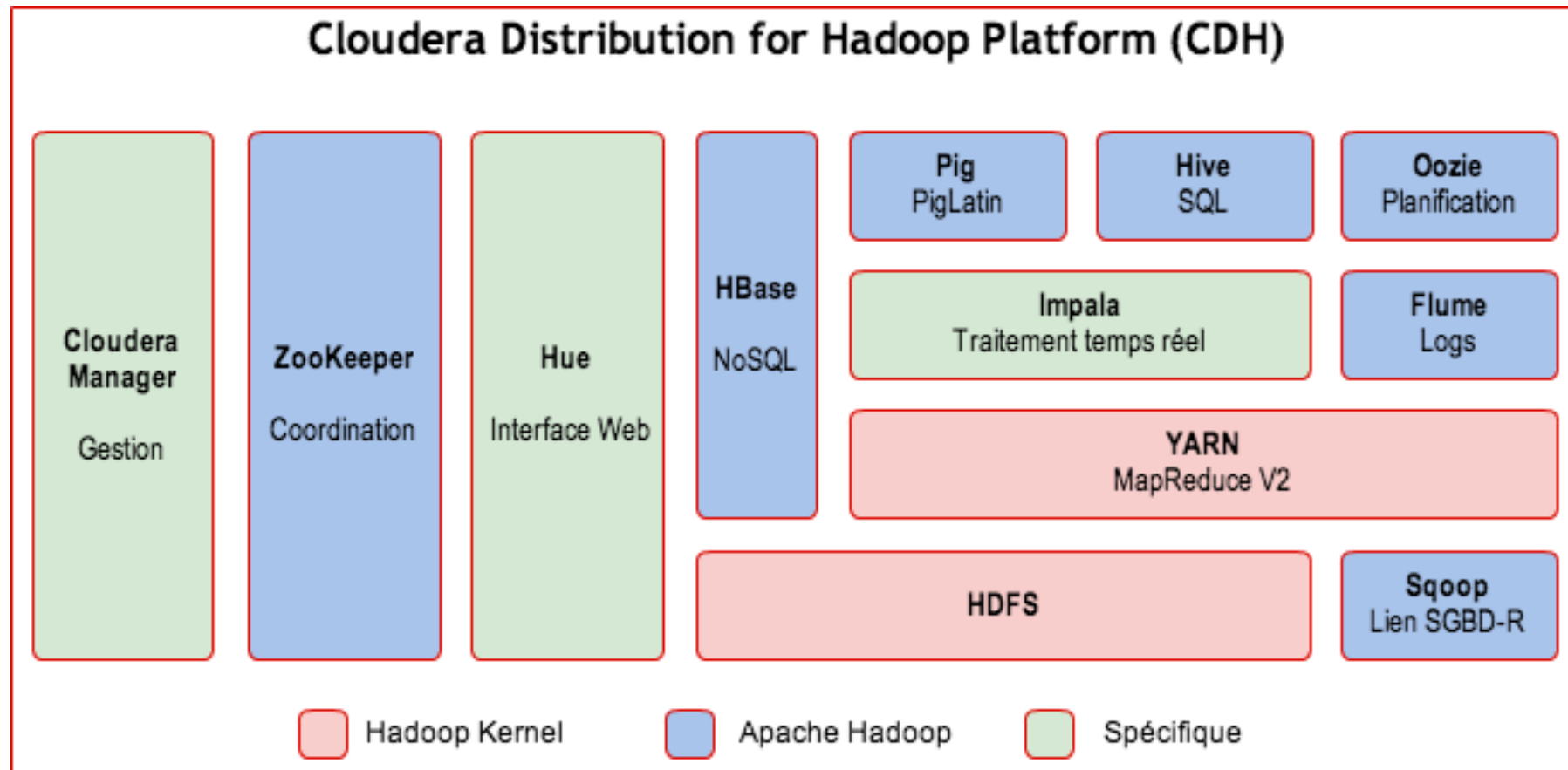
Cloudera CDH

The Cloudera logo, consisting of the word "CLOUDERA" in orange capital letters, is displayed inside a white rectangular box with a subtle drop shadow.

CLOUDERA

- **CDH : Cloudera's Distribution including Apache Hadoop**
- Dans le cadre de notre cours , nous utilisons la version gratuite de la **distribution Cloudera (sous forme de machine virtuelle)**.
- Elle nous permet de bénéficier d'un **cluster simple nœud** prêt à l'emploi
- Elle fournit également, des **outils d'administration** supplémentaires qui facilitent son usage.

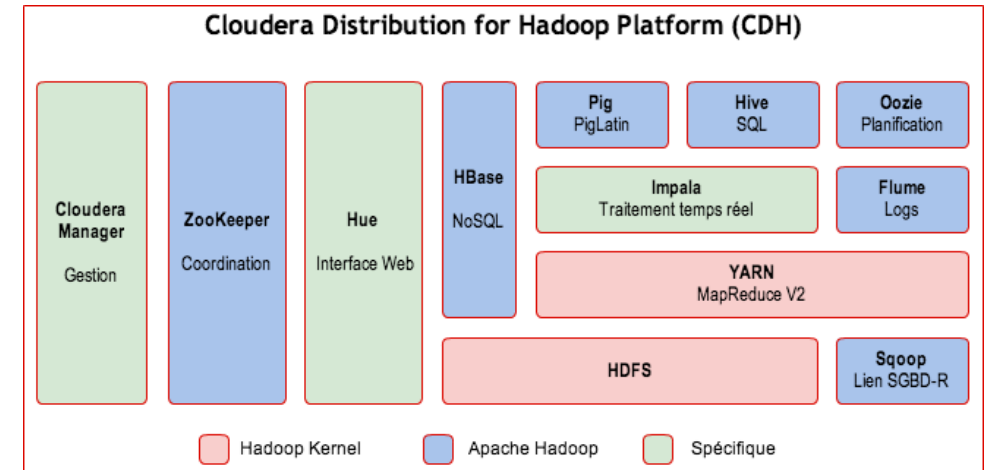
Les composants de CDH (1/2)



Les composants de CDH (2/2)

Composants d'origine Apache

- **HDFS** : File System distribué.
- **MapReduce** : Framework de traitement parallélisé.
- **HBase** : Base de données NoSQL (accès read/write aléatoires).
- **Hive** : Requêtage de type SQL.
- **Pig** : Scripting et requêtage Hadoop.
- **Oozie** : Workflow et planification de jobs Hadoop.
- **Sqoop** : Intégration de bases SQL.
- **Flume** : Exploitation de fichiers (log) dans Hadoop.
- **ZooKeeper** : Service de coordination pour les applications distribuées.



Composants d'origine Cloudera :

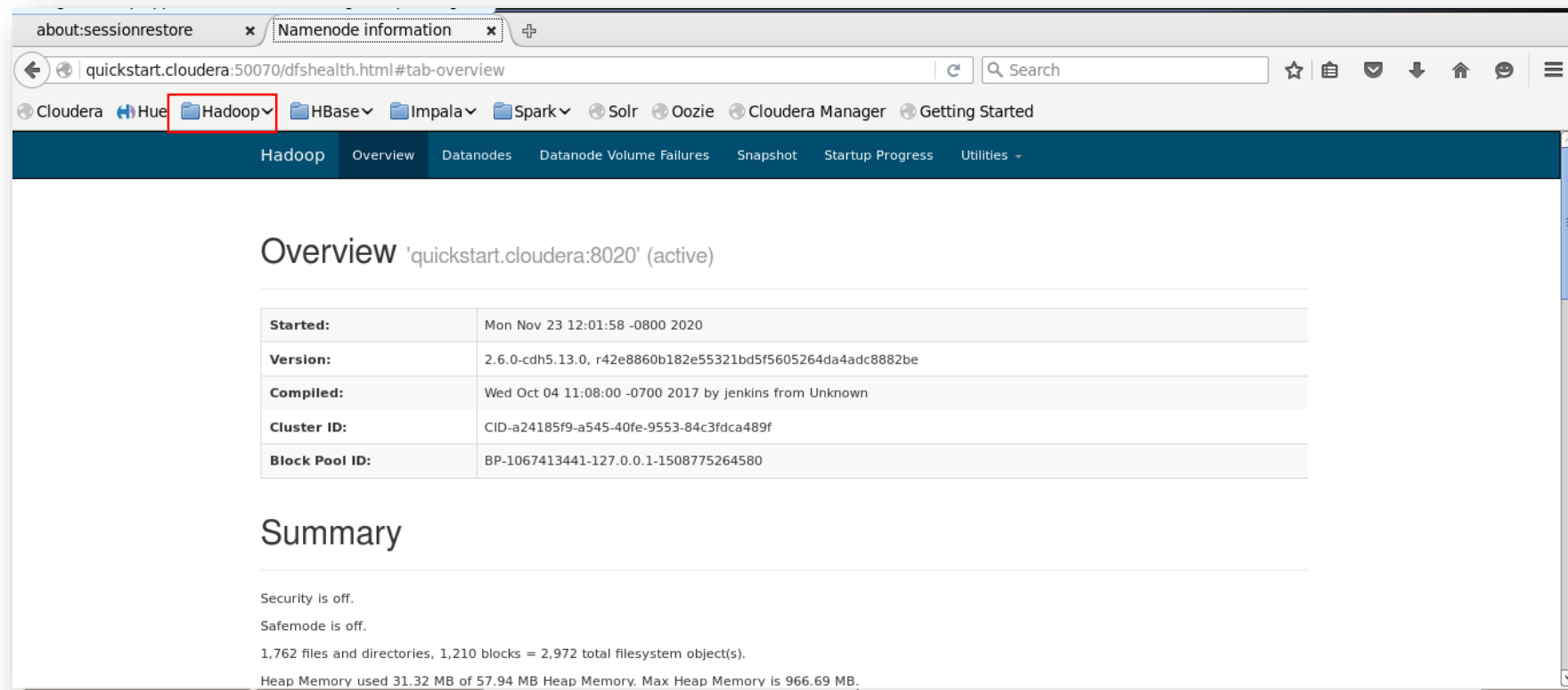
- **Hue** : un ensemble d'applications web permettant d'interagir avec un cluster Apache Hadoop.
- **Cloudera Impala** : permet la gestion en temps réel des requêtes sur les données stockées sur les clusters Hadoop
- **Cloudera Manager** : Déploiement et gestion des composants Hadoop.

Interfaces utilisateur (1/4)

- Hadoop est essentiellement piloté par des outils consoles (CLI), mais aussi des interfaces utilisateurs sont mises à disposition de l'utilisateur par cloudera:
- **NameNode** dispose d'une interface web (via un serveur web intégré) qui permet de parcourir les fichiers et répertoires stockés sur HDFS.
- Le **DataNode**, le **Secondary NameNode**, le **ResourceManager** et le **NodeManager** disposent également chacun d'une interface utilisateur.

Interfaces utilisateur (2/4)

- NameNode



The screenshot shows the Cloudera Hue web interface. The browser address bar displays 'quickstart.cloudera:50070/dfshealth.html#tab-overview'. The navigation bar includes links for Cloudera, Hue, Hadoop (highlighted with a red box), HBase, Impala, Spark, Solr, Oozie, Cloudera Manager, and Getting Started. The main content area is titled 'Overview 'quickstart.cloudera:8020' (active)'. It contains a table with the following information:

Started:	Mon Nov 23 12:01:58 -0800 2020
Version:	2.6.0-cdh5.13.0, r42e8860b182e55321bd5f5605264da4adc8882be
Compiled:	Wed Oct 04 11:08:00 -0700 2017 by jenkins from Unknown
Cluster ID:	CID-a24185f9-a545-40fe-9553-84c3fdca489f
Block Pool ID:	BP-1067413441-127.0.0.1-1508775264580

Below the table is a 'Summary' section with the following text:

Security is off.
Safemode is off.
1,762 files and directories, 1,210 blocks = 2,972 total filesystem object(s).
Heap Memory used 31.32 MB of 57.94 MB Heap Memory. Max Heap Memory is 966.69 MB.

Interfaces utilisateur (3/4)

- Ressource Manager

quickstart.cloudera:8088/cluster

Cloudera Hue Hadoop HBase Impala Spark Solr Oozie Cloudera Manager Getting Started

Logged in as: dr.who

All Applications

Cluster

- About
- Nodes
- Applications
- NEW
- NEW SAVING
- SUBMITTED
- ACCEPTED
- RUNNING
- FINISHED
- FAILED
- KILLED
- Scheduler

Cluster Metrics

Apps Submitted	Apps Pending	Apps Running	Apps Completed	Containers Running	Memory Used	Memory Total	Memory Reserved	VCores Used	VCores Total	VCores Reserved
0	0	0	0	0	0 B	8 GB	0 B	0	8	0

Cluster Nodes Metrics

Active Nodes	Decommissioning Nodes	Decommissioned Nodes	Lost Nodes	Unhealthy Nodes	Rebooted Nodes
1	0	0	0	0	0

User Metrics for dr.who

Apps Submitted	Apps Pending	Apps Running	Apps Completed	Containers Running	Containers Pending	Containers Reserved	Memory Used	Memory Pending	Memory Reserved	VCores Used	VCores Pending	VCores Reserved
0	0	0	0	0	0	0	0 B	0 B	0 B	0	0	0

Show 20 entries

Search:

ID	User	Name	Application Type	Queue	StartTime	FinishTime	State	FinalStatus	Running Containers	Allocated CPU VCores	Allocated Memory MB	Reserved CPU VCores	Reserved Memory MB	Progress	Tracking UI
No data available in table															

Showing 0 to 0 of 0 entries

First Previous Next Last

Interfaces utilisateur (4/4)

- **Cloudera** inclut aussi **Hue** : une interface web open-source sous licence Apache prenant en charge **Hadoop** et son **écosystème**.
- **Hue** agrège les composants Apache Hadoop les plus courants en une seule interface en cherchant à améliorer l'expérience utilisateur.
- Son objectif principal est de permettre aux utilisateurs de ne pas se soucier de la complexité sous-jacente d'Hadoop et à limiter l'utilisation de ligne de commande.



Manipulation du contenu Hadoop (1/8)

- Il est néanmoins vital de savoir de servir des commandes de base (qui pourraient par exemple être appelées depuis des scripts shell).
- Il existe différentes façons d'interagir avec un système de fichiers HDFS telles que des **API Java**, des **protocoles réseaux classiques** (HTTP, FTP, WebHDFS, HttpFS...), des **protocoles propriétaires** (Amazon S3) ou tout simplement via la **ligne de commandes**. **C'est ce dernier moyen d'interaction que nous allons détailler dans la suite de cette section.**
- **HDFS s'appuie sur la spécification POSIX** pour son système de fichiers. Il permet entre autres **de pouvoir manipuler des** fichiers et de disposer de droits sur ces fichiers. **Notons que HDFS n'implémente pas totalement la spécification POSIX**
- Toutes les commandes supportées par HDFS sont disponibles via la commande suivante :

\$ `hadoop fs`

Manipulation du contenu Hadoop (2/8)

```
$ hadoop fs
```

```
Usage: hadoop fs [generic options]
```

```
[-cat [-ignoreCrc] <src> ...]  
[-chgrp [-R] GROUP PATH...]  
[-chmod [-R] <MODE[,MODE]... | OCTALMODE> PATH...]  
[-chown [-R] [OWNER][:[GROUP]] PATH...]  
[-copyFromLocal [-f] [-p] <localsrc> ... <dst>]  
[-copyToLocal [-p] [-ignoreCrc] [-crc] <src> ... <localdst>]  
[-count [-q] <path> ...]  
[-cp [-f] [-p] <src> ... <dst>]  
[-df [-h] [<path> ...]]  
[-du [-s] [-h] <path> ...]  
[-get [-p] [-ignoreCrc] [-crc] <src> ... <localdst>]  
[-help [cmd ...]]  
[-ls [-d] [-h] [-R] [<path> ...]]  
[-mkdir [-p] <path> ...]  
[-moveFromLocal <localsrc> ... <dst>]  
[-moveToLocal <src> <localdst>]  
[-mv <src> ... <dst>]  
[-put [-f] [-p] <localsrc> ... <dst>]  
[-rm [-f] [-r|-R] [-skipTrash] <src> ...]  
[-rmdir [--ignore-fail-on-non-empty] <dir> ...]  
[-tail [-f] <file>]
```

Manipulation du contenu Hadoop (3/8)

- Il s'agit des commandes de bases qu'un utilisateur d'un **système POSIX**, Linux par exemple.
- Nous dénoterons toutefois deux principales différences :
 - La première différence concerne l'appel à une commande HDFS qui doit se faire comme une sous-commande de la commande Hadoop.
 - La seconde différence concerne **l'absence de répertoire courant**. Il n'y a donc pas de possibilité d'utiliser la **commande cd** puisqu'elle **n'existe pas**. Par conséquent tous les chemins sont **absolus** ou **relatifs** par rapport au répertoire de l'utilisateur en cours, dans notre cas il s'agira de **/user/cloudera**. Pour le **chemin absolu**, deux écritures sont autorisées :
 - une écriture via une **URL**, par exemple, **hdfs://quickstart.cloudera:8022/monDossier/premierfichier.txt**
 - une écriture via une **URI** (Uniform Resource Identifier), par exemple, **/monDossier/premierfichier.txt**. La localisation du namenode **hdfs://quickstart.cloudera:8022** est définie par la valeur de la propriété **fs.defaultFS** (**paramètre présent dans le fichier core-site.xml**). Pour afficher ce fichier Taper :

```
gedit /etc/hadoop/conf/core-site.xml&
```

À noter que si un chemin absolu avec une URI est utilisé, il sera complété par la localisation du nœud namenode

Manipulation du contenu Hadoop (4/8)

Lecture du contenu d'un répertoire

- La lecture d'un fichier se fera par la sous commande **-ls**.
- Nous donnons ci-dessous un exemple qui liste le contenu du répertoire racine en utilisant un chemin absolu via une **URI** /.

```
[cloudera@quickstart ~]$ hadoop fs -ls /  
Found 6 items  
drwxrwxrwx - hdfs supergroup          0 2017-10-23 09:15 /benchmarks  
drwxr-xr-x - hbase supergroup          0 2020-11-29 08:56 /hbase  
drwxr-xr-x - solr solr                  0 2017-10-23 09:18 /solr  
drwxrwxrwt - hdfs supergroup          0 2020-05-19 09:17 /tmp  
drwxr-xr-x - hdfs supergroup          0 2017-10-23 09:17 /user  
drwxr-xr-x - hdfs supergroup          0 2017-10-23 09:17 /var
```

- Ci-dessous nous obtenons le même résultat en utilisant cette fois un chemin absolu via une **URL** **hdfs://quickstart.cloudera:8022/**.

```
[cloudera@quickstart ~]$ hadoop fs -ls hdfs://quickstart.cloudera:8022/  
Found 6 items  
drwxrwxrwx - hdfs supergroup          0 2017-10-23 09:15 hdfs://quickstart.cloudera:8022/benchmarks  
drwxr-xr-x - hbase supergroup          0 2020-11-29 08:56 hdfs://quickstart.cloudera:8022/hbase  
drwxr-xr-x - solr solr                  0 2017-10-23 09:18 hdfs://quickstart.cloudera:8022/solr  
drwxrwxrwt - hdfs supergroup          0 2020-05-19 09:17 hdfs://quickstart.cloudera:8022/tmp  
drwxr-xr-x - hdfs supergroup          0 2017-10-23 09:17 hdfs://quickstart.cloudera:8022/user  
drwxr-xr-x - hdfs supergroup          0 2017-10-23 09:17 hdfs://quickstart.cloudera:8022/var
```

Manipulation du contenu Hadoop (5/8)

Copie de fichiers (1/2)

- La copie de fichiers est **similaire** à ce que nous pouvons réaliser depuis un système de **fichiers distant** comme par exemple **FTP**. Ainsi, la **copie** de fichiers **vers le système de fichiers HDFS depuis le système local**, se fait via la commande **-put** ou son alias **-copyFromLocal**. La **copie** de fichiers **vers le système local depuis le système de fichiers HDFS** se fait via la commande **-get** ou son alias **-copyToLocal**.
- À noter que les commandes **-moveFromLocal** et **-moveToLocal** **copieront** les fichiers depuis ou vers le système de fichiers HDFS **et supprimeront les fichiers sources**.

Manipulation du contenu Hadoop (6/8)

Copie de fichiers (2/2)

- Ci-dessous nous donnons un exemple permettant de copier un fichier depuis le système de fichiers local vers le système de fichiers HDFS

```
[cloudrea@quikstart~]$ hadoop fs -put /home/cloudera/ReadMe.txt README
```

```
[cloudrea@quikstart~]$ hadoop fs -ls /user/cloudera
```



```
cloudera@quickstart:~  
File Edit View Search Terminal Help  
[cloudera@quickstart ~]$ hadoop fs -ls /user/cloudera  
Found 7 items  
drwxr-xr-x - cloudera cloudera 0 2016-09-21 05:14 /user/cloudera/.Trash  
drwxr-xr-x - cloudera cloudera 0 2016-03-10 14:31 /user/cloudera/.sparkStaging  
-rw-r--r-- 1 cloudera cloudera 10 2016-09-21 05:27 /user/cloudera/README  
drwxr-xr-x - cloudera cloudera 0 2016-09-21 04:56 /user/cloudera/myTestDir  
drwxr-xr-x - cloudera cloudera 0 2016-09-21 05:20 /user/cloudera/myTestDir2  
drwxr-xr-x - cloudera cloudera 0 2016-03-19 18:22 /user/cloudera/output  
drwxr-xr-x - cloudera cloudera 0 2016-03-19 18:16 /user/cloudera/wordcount  
[cloudera@quickstart ~]$
```

- Ci-dessous nous donnons un exemple permettant de copier un fichier depuis le système de fichiers HDFS vers le système de fichiers local.

```
[cloudrea@quikstart~]$ hadoop fs -get README /home/cloudera/readme-copy.txt
```


Manipulation du contenu Hadoop (7/8)

Remarque

- Les informations retournées par la sous commande `–ls` sont très **similaires** à ce qu'on obtient avec la commande **Linux ls -l**, à part une **différence** notable qui vient de **la seconde colonne donnant le facteur de réplication du fichier considéré**. Dans notre cas, celui-ci vaut **1** puisque nous avons configuré un cluster avec un nœud unique.
- La deuxième colonne d'un `ls -l` depuis un système **Linux correspond au nombre de liens physiques et non au nombre de facteurs de réplication du fichier**.

Manipulation du contenu Hadoop (8/8)

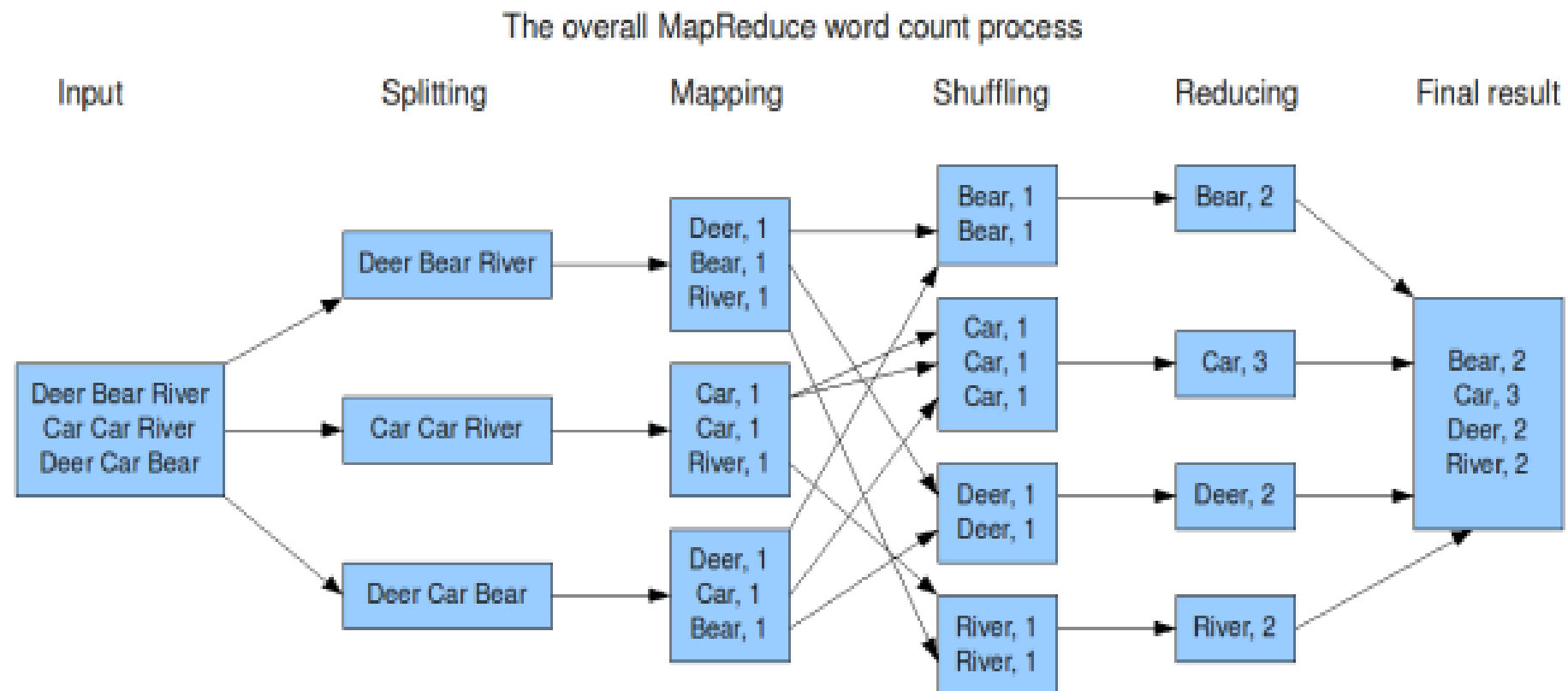
Suppression de fichiers

- Dans l'exemple qui suit, nous montrons comment supprimer un fichier depuis le système de fichiers HDFS avec la commande -rm.

```
[cloudrea@quikstart~]$ hadoop fs -rm README
```

Programmation Hadoop : WordCount

Rappel : Job MapReuce (WordCount)



Programmation Hadoop : Trois classes

- Comme indiqué précédemment, Hadoop est développé en **Java**. Les tâches MAP/REDUCE sont donc implémentées par le biais d'interfaces Java (il existe cependant des **wrappers** très simples permettant d'implémenter ses tâches dans n'importe quel langage).
- Un programme Hadoop se compile au sein d'un **.jar**.
- Pour développer un programme Hadoop, on va créer **trois classes distinctes** :
 - **Une classe dite « Driver »** qui contient la **fonction main** du programme. Cette classe se chargera d'informer Hadoop des **types de données clef/valeur** utilisées, des classes se chargeant des opérations **MAP** et **REDUCE**, et des **fichiers** HDFS à utiliser pour les **entrées/sorties**.
 - **Une classe MAP** (qui effectuera l'opération MAP).
 - **Une classe REDUCE** (qui effectuera l'opération REDUCE).

Programmation Hadoop – Classe Driver (1/11)

- La classe Driver contient la fonction « main » de notre programme. Au sein du main() en question, on va effectuer les opérations suivantes :
 1. Créer un objet Configuration de Hadoop, qui est nécessaire pour permettre à Hadoop d'obtenir la configuration générale du cluster. L'objet en question pourrait aussi nous permettre de récupérer nous-même des options de configuration qui nous intéressent.
 2. Permettre à Hadoop de récupérer d'éventuels arguments génériques disponibles sur la ligne de commande (par exemple le nom du package de la tâche à exécuter si le .jar en contient plusieurs). On va également récupérer les arguments supplémentaires pour s'en servir; on souhaite que l'utilisateur puisse préciser le nom du fichier d'entrée et le nom du répertoire de sortie HDFS pour nos tâches Hadoop grâce à la ligne de commande.
 3. Créer un nouvel objet Hadoop Job, qui désigne une tâche Hadoop
 4. Utiliser cet objet Job pour informer Hadoop du nom de nos classes Driver, MAP et REDUCE.
 5. Utiliser le même objet pour informer Hadoop des types de données utilisés dans notre programme pour les couples (clef;valeur) MAP et REDUCE.
 6. Informer Hadoop des fichiers d'entrée/sortie pour notre tâche sur HDFS.
 7. Enfin, utiliser l'objet Job créé précédemment pour déclencher le lancement de la tâche via le cluster Hadoop.

Programmation Hadoop – Classe Driver (2/11)

On reprend ici l'exemple du compteur d'occurrences de mots décrit précédemment.

- Le prototype de notre fonction **main()** :

```
public static void main(String[] args) throws Exception
```

- On se sert de **args** pour récupérer les arguments de la ligne de commande. Plusieurs fonctions Hadoop appelées au sein du **main** sont susceptibles de déclencher des exceptions – on l'indique donc lors de la déclaration.
- Avant toute chose, on crée dans notre main **un nouvel objet Configuration Hadoop** :

```
// Créé un objet de configuration Hadoop.  
Configuration conf=new Configuration();
```

- Le package à importer est: **org.apache.hadoop.conf.Configuration**

Programmation Hadoop – Classe Driver (3/11)

- Ensuite, on passe à Hadoop les arguments de la ligne de commande pour lui permettre de récupérer ceux qui sont susceptibles de lui être adressés :

```
String[] ourArgs =  
new GenericOptionsParser(conf, args).getRemainingArgs();
```

- On utilise pour ce faire **un objet Hadoop GenericOptionsParser**, dont le **package** est : **org.apache.hadoop.util.GenericOptionsParser**
- La fonction **getRemainingArgs()** de notre objet nous permet par ailleurs de récupérer les arguments non exploités par Hadoop au sein d'un tableau **ourArgs** – ce qui nous permettra de les utiliser par la suite.
- On peut ainsi rendre paramétrable facilement une tâche Hadoop lors de l'exécution, par le biais des arguments de la ligne de commande.

Programmation Hadoop – Classe Driver (4/11)

- On crée ensuite un nouvel objet Hadoop Job :

```
Job job=Job.getInstance(conf, "Compteur de mots v1.0");
```

- Le package à importer est le suivant: **org.apache.hadoop.mapreduce.Job**
- On passe au constructeur : notre objet **Configuration**, ainsi qu'une **description textuelle** courte du programme Hadoop.
- Ensuite, il faut indiquer à Hadoop – par le biais de l'objet **Job** nouvellement créé – quelles sont les **classes Driver, Map et Reduce** de notre programme Hadoop.
- Dans notre cas, il s'agira respectivement des classes **WCount, WCountMap et WCountReduce** du package **com.fsb.mapreduce**.

Programmation Hadoop – Classe Driver (5/11)

- On utilise pour ce faire les fonctions suivantes:

```
job.setJarByClass(WCount.class);  
job.setMapperClass(WCountMap.class);  
job.setReducerClass(WCountReduce.class);
```

- Il faut ensuite indiquer à Hadoop quels sont les types de données que l'on souhaite utiliser pour **les couples (clef;valeur)** de nos opérations **map et reduce**. Dans le cas de notre compteur d'occurrences de mots, on souhaite utiliser des chaînes de caractères pour les clefs (nos mots) et des entiers pour nos occurrences.

Remarque: on ne doit pas utiliser les types classiques **Int** et **String** de Java pour désigner nos types, mais des classes qui leur correspondent et qui sont propres à Hadoop. Dans notre cas, les classes **IntWritable** et **Text**.

Programmation Hadoop – Classe Driver (6/11)

```
job.setOutputKeyClass(Text.class);  
job.setOutputValueClass(IntWritable.class);
```

- Les packages des types en question:
 - `org.apache.hadoop.io.IntWritable` et `org.apache.hadoop.io.Text`
- Il en existe beaucoup d'autres dans **`org.apache.hadoop.io.*`**.
- Ensuite, on doit indiquer où se situent nos données d'entrée et de sortie dans HDFS. On utilise pour ce faire les classes Hadoop **`FileInputFormat`** et **`FileOutputFormat`**.
- Ces classes sont implémentées suivant un **design pattern Singleton** – il n'est pas nécessaire de les instancier dans notre cas

Programmation Hadoop – Classe Driver (7/11)

- On procède de la manière suivante :

```
FileInputFormat.addInputPath(job, new Path(ourArgs[0]));  
FileOutputFormat.setOutputPath(job, new Path(ourArgs[1]));
```

- Les packages à utiliser :
 - **org.apache.hadoop.mapreduce.lib.input.FileInputFormat**
 - **org.apache.hadoop.mapreduce.lib.output.FileOutputFormat**
 - **org.apache.hadoop.fs.Path**
- On utilise les arguments restants après ceux qu'a utilisé Hadoop. Le code est ici simplifié – on devrait en théorie vérifier la taille du tableau ourArgs pour éviter d'éventuelles erreurs.

Programmation Hadoop – Classe Driver (8/11)

- Enfin, il reste à lancer l'exécution de la tâche par le biais du cluster Hadoop. On procède ainsi:

```
if(job.waitForCompletion(true))  
    System.exit(0);  
System.exit(-1);
```

- La fonction **waitForCompletion** de l'objet job va exécuter la tâche et attendre la fin de son exécution. Elle prend un argument: **un booléen indiquant à Hadoop si oui ou non** il doit donner des indications sur la progression de l'exécution à l'utilisateur sur la sortie standard (stdout).
- Elle renvoie **true en cas de succès**; ici, on terminera l'exécution du programme en **renvoyant 0 si tout s'est bien passé, et -1 en cas de problème** (codes de retour unix standards).

Programmation Hadoop – Classe Driver (9/11)

- La classe Driver contient la fonction « main » de notre programme.
- Le code complet de notre classe Driver (1/3):

```
package hadoop.wordcount;

import org.apache.hadoop.fs.Path;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.util.GenericOptionsParser;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.io.IntWritable;

// Notre classe Driver (contient le main du programme Hadoop).
public class WCount
{
```

Programmation Hadoop – Classe Driver (10/11)

- Le code complet de notre classe Driver (2/3):

```
// Le main du programme.
public static void main(String[] args) throws Exception
{
    // Créé un object de configuration Hadoop.
    Configuration conf=new Configuration();

    // Permet à Hadoop de lire ses arguments génériques,
    // récupère les arguments restants dans ourArgs.
    String[] ourArgs=new GenericOptionsParser(conf,
                                                args).getRemainingArgs();

    // Obtient un nouvel objet Job: une tâche Hadoop. On
    // fourni la configuration Hadoop ainsi qu'une description
    // textuelle de la tâche.
    Job job=Job.getInstance(conf, "Compteur de mots v1.0");
```

Programmation Hadoop – Classe Driver (11/11)

- Le code complet de notre classe Driver (3/3):

```
// Défini les classes driver, map et reduce.
job.setJarByClass(WCount.class);
job.setMapperClass(WCountMap.class);
job.setReducerClass(WCountReduce.class);

// Défini types clefs/valeurs de notre programme Hadoop.
job.setOutputKeyClass(Text.class);
job.setOutputValueClass(IntWritable.class);

// Défini les fichiers d'entrée du programme et le
// répertoire des résultats. On se sert du premier et du
// deuxième argument restants pour permettre à
// l'utilisateur de les spécifier lors de l'exécution.
FileInputFormat.addInputPath(job, new Path(ourArgs[0]));
FileOutputFormat.setOutputPath(job, new Path(ourArgs[1]));

// On lance la tâche Hadoop. Si elle s'est effectuée
// correctement, on renvoie 0. Sinon, on renvoie -1.
if(job.waitForCompletion(true))
    System.exit(0);
System.exit(-1);
}
```

Programmation Hadoop – Classe MAP (1/6)

- La classe **MAP** va être en charge de l'**opération MAP** de notre programme.
- Elle doit étendre la classe Hadoop : **org.apache.hadoop.mapreduce.Mapper**. Il s'agit d'une classe générique qui est paramétrée **avec quatre types** :
 - **Un type keyin**: le type de clef d'entrée.
 - **Un type valuein**: le type de valeur d'entrée.
 - **Un type keyout**: le type de clef de sortie.
 - **Un type valueout**: le type de valeur de sortie.
- Le type **keyin** est notamment utile lorsqu'on utilise des fonctionnalités plus avancées, comme la possibilité d'effectuer plusieurs opérations **MAP** les unes à la suite des autres, auquel cas notre opération **map** recevra en entrée des **couples (clef;valeur)**.

Programmation Hadoop – Classe MAP (2/6)

- Dans notre exemple, notre classe **Map** sera déclarée ainsi :

```
public class WCountMap extends Mapper<Object, Text, Text,  
IntWritable>
```

- On utilise ici comme types:
 - **Text** pour le type **valuein**, puisque **notre valeur d'entrée** à la fonction **Map** est **une chaîne de caractères** (une ligne de texte).
 - **Text** pour le type **keyout**, puisque **notre valeur de clef pour les couples (clef;valeur)** de la fonction **Map** est également **une chaîne de caractères** (le mot dont on compte les occurrences).
 - **IntWritable** pour le type **valueout**, puisque **notre valeur pour les couples (clef;valeur)** de la fonction **Map** est un entier (le nombre d'occurrences).

Remarque : Ici aussi, on utilise les types Hadoop et non les types natifs Java (Int et String).

Programmation Hadoop – Classe MAP (3/6)

- Au sein de la classe **Mapper**, c'est la fonction **map** qui va s'occuper d'effectuer la **tâche MAP**. C'est la seule qu'on doit absolument implémenter.
- Elle prend trois arguments: la clef d'entrée **keyin** (qu'on ignore dans notre cas), la valeur d'entrée **valuein** (la ligne de texte dont on souhaite compter les mots), et un **Context** Java qui représente un **handle Hadoop** et nous permettra de retourner les couples (**clef;valeur**) résultant de notre opération Map.
- Le prototype de notre fonction map :

```
protected void map(Object key,Text value,Context context)  
    throws IOException, InterruptedException
```

- Comme pour la fonction main, la fonction map appellera des fonctions **susceptibles de déclencher des exceptions** (notamment concernant l'interruption de l'exécution Hadoop ou des problèmes d'accès HDFS) – on le précise donc dans sa déclaration.

Programmation Hadoop – Classe MAP (4/6)

- Au sein de la méthode **map**, on va donc effectuer la tâche **MAP** de notre programme **MAP/REDUCE**.
- Dans le cadre de notre exemple, la fonction devra parcourir **la ligne de texte fournie en entrée**, et **renvoyer un couple (clef;valeur)** pour chacun des mots. Ce couple devra avoir pour **clef le mot en question**, et pour **valeur l'entier « 1 »**.
- Dans la fonction **map**, afin d'indiquer à Hadoop qu'on souhaite renvoyer **un couple (clef;valeur)**, on utilise la fonction **write** de notre objet **Context**. Elle peut être appelée autant de fois que nécessaire; **une fois pour chacun des couples (clef;valeur)** qu'on souhaite renvoyer. Par exemple :

```
context.write("ciel", 1);
```
- Il faut évidemment que **la clef et la valeur renvoyées** ainsi correspondent aux types **keyout** et **valueout** de notre classe **Mapper**

Programmation Hadoop – Classe MAP (5/6)

Notre classe Map d'exemple en intégralité : (1/2)

```
package hadoop.wordcount;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.io.IntWritable;
import java.util.StringTokenizer;
import org.apache.hadoop.mapreduce.Mapper;
import java.io.IOException;

// Notre classe MAP.
public class WCountMap extends Mapper<Object, Text, Text, IntWritable>
{
    // IntWritable contenant de valeur 1.
    private static final IntWritable ONE=new IntWritable(1);
```

Programmation Hadoop – Classe MAP (6/6)

Notre classe Map d'exemple en intégralité : (2/2)

```
// La fonction MAP elle-même.
protected void map(Object offset, Text value, Context context)
    throws IOException, InterruptedException
{
    // Un StringTokenizer va nous permettre de parcourir chacun des
    // mots de la ligne qui est passée à notre opération MAP.
    StringTokenizer tok=new StringTokenizer(value.toString(), " ");

    while(tok.hasMoreTokens())
    {
        Text word=new Text(tok.nextToken());
        // On renvoie notre couple (clef;valeur): le mot courant suivi
        // de la valeur 1 (définie dans la constante ONE).
        context.write(word, ONE);
    }
}
```

Programmation Hadoop – Classe Reduce (1/6)

- La classe **REDUCE** va être en charge de l'opération **REDUCE** de notre programme.
- Elle doit étendre la classe **Hadoop org.apache.hadoop.mapreduce.Reducer**. Il s'agit là aussi d'une classe générique qui est paramétrée avec les mêmes quatre types que pour la classe Mapper: **keyin, valuein, keyout et valueout**.
- On rappelle que l'opération **REDUCE** recevra en entrée **une clef unique, associée à toutes les valeurs pour la clef en question**.
- Dans le cas **du compteur d'occurrences de mots**, on recevra en entrée **une valeur unique pour la clef**, par exemple « **ciel** », suivi de toutes **les valeurs** qui ont été rencontrées à la sortie de l'opération **MAP** pour la clef « **ciel** » (par exemple cinq fois la valeur « **1** » si le mot « **ciel** » était présent cinq fois dans notre texte d'exemple).

Programmation Hadoop – Classe Reduce (2/6)

- Dans notre exemple, notre classe Reduce sera définie comme suit:

```
public class WCountReduce extends Reducer<Text,  
IntWritable, Text, Text>
```

- On utilise pour chacun des types paramétrables:
 - **Text pour keyin:** il s'agit de **notre clef unique** d'entrée – **le mot concerné**.
 - **IntWritable pour valuein:** le type **de nos valeurs associées à cette clef** (le nombre d'occurrences, un entier).
 - **Text pour keyout:** le type de **clef de sortie**. Nous ne modifierons pas la clef, il s'agira toujours du mot unique concerné – on utilise donc Text.
 - **Text pour valueout:** le type de **valeur de sortie**. On utilise ici Text – on renverra le nombre total d'occurrences pour le mot concerné sous la forme d'une chaîne de caractères (on pourrait également utiliser IntWritable ici).
- Là aussi, on utilise **les types de données propres à Hadoop**.

Programmation Hadoop – Classe Reduce (3/6)

- Au sein de la classe **Reducer**, c'est la fonction **reduce** qui va effectuer l'opération **REDUCE**. C'est la seule qu'on doive implémenter.
- Elle prend trois arguments: **la clef concernée**, **un Iterable java (une liste)** de toutes les valeurs qui lui sont associées et qui ont été renvoyées par l'opération MAP, et enfin **un objet Context** java similaire à celui de la fonction map de la classe Mapper, et qui nous permettra **de renvoyer notre valeur finale, associée à la clef**.
- Dans notre exemple, la déclaration de la fonction reduce:

```
public void reduce(Text key, Iterable<IntWritable>  
                    values, Context context)  
                    throws IOException, InterruptedException
```

- Comme pour map, la fonction fait appel à des fonctions Hadoop susceptibles de provoquer des exceptions – on l'indique ici aussi.

Programmation Hadoop – Classe Reduce (4/6)

- Au sein de la fonction **reduce**, on pourra renvoyer **un couple (clef;valeur)** en résultat exactement de la même manière que pour la fonction map, par le biais d'un appel à la fonction **write de notre objet Context**.
- Par exemple :

```
context.write("ciel", "5 occurrences");
```

- Contrairement à la fonction **map**, en revanche, on cherche à ne produire qu'une et une seule valeur de retour pour la clef concernée. On n'appellera donc la fonction **write** qu'une seule fois.

Programmation Hadoop – Classe Reduce (5/6)

Notre classe Reduce d'exemple en intégralité : (1/2)

```
package hadoop.wordcount;

import org.apache.hadoop.io.Text;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.mapreduce.Reducer;
import java.util.Iterator;
import java.io.IOException;

// Notre classe REDUCE - paramétrée avec un type Text pour la clef, un
// type de valeur IntWritable, et un type de retour (le retour final de
// la fonction Reduce) Text.
public class WCountReduce extends Reducer<Text, IntWritable, Text, Text>
{
```

Programmation Hadoop – Classe Reduce (6/6)

Notre classe Reduce d'exemple en intégralité : (2/2)

```
// La fonction REDUCE elle-même. Les arguments: la clef key, un
// Iterable de toutes les valeurs qui sont associées à la clef en
// question, et le contexte Hadoop (un handle qui nous permet de
// renvoyer le résultat à Hadoop).
public void reduce(Text key, Iterable<IntWritable> values,
                  Context context)
    throws IOException, InterruptedException
{
    // Pour parcourir toutes les valeurs associées à la clef fournie.
    Iterator<IntWritable> i=values.iterator();
    int count=0; // Notre total pour le mot concerné.
    while(i.hasNext()) // Pour chaque valeur...
        count+=i.next().get(); // ...on l'ajoute au total.
    // On renvoie le couple (clef;valeur) constitué de notre clef key
    // et du total, au format Text.
    context.write(key, new Text(count+" occurrences."));
}
```

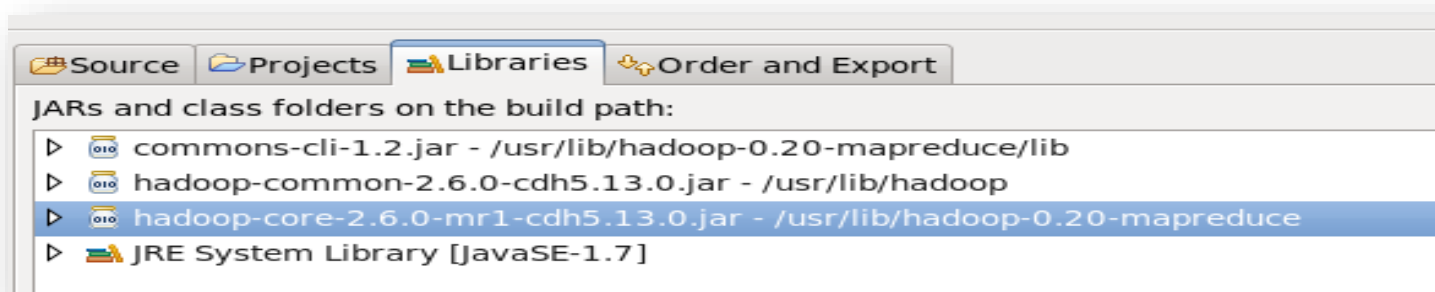
Programmation Hadoop

Remarques :

- Hadoop est une plate-forme récente, en développement constant: l'API Java change régulièrement. Il faut toujours s'informer sur les dernières modifications afin de rester à jour des dernières modifications. Par ailleurs, la distribution Hadoop de cloudera comprend de nombreux exemples de programmes Map/Reduce, mis à jour afin d'utiliser l'API la plus récente.
- Même si l'API change régulièrement, les anciennes manières de procéder restent généralement disponibles pendant plusieurs mois après une mise à jour.
- Enfin, il existe de nombreuses autres possibilités offertes par l'API: extraire des paramètres de configurations globaux du cluster, implémenter sa propre classe `FileInputFormat` pour des traitements plus complexes sur les données d'entrée, etc... d'une manière générale, se référer à la documentation de l'API Hadoop: <https://hadoop.apache.org/docs/>

Programmation Hadoop : Compilation

- Pour compiler un programme Hadoop, il suffit d'utiliser le compilateur **javac** comme pour tout autre programme Java. (Dans la cas de notre cours, nous utiliserons **l'IDE Eclipse** pour la compilation)
- Il est cependant nécessaire d'inclure les librairies appropriées. Elles sont disponibles avec les distributions de Hadoop, **sous la forme de fichiers .jar**. Les librairies principales à utiliser dans notre cas :



- ... il en existe d'autres pour des cas d'utilisation plus complexes, à inclure en fonction des besoins.
- Enfin, après compilation, on package le programme Hadoop à l'intérieur d'un fichier **.jar** pour son exécution.

Programmation Hadoop : Exécution

- Pour exécuter un programme Hadoop, on utilise là aussi le programme en ligne de commande `hadoop`. La syntaxe est la suivante :

```
hadoop jar [JARFILE] [DRIVERCLASS] [PARAMETERS]
```

- Par exemple pour notre compteur d'occurrences de mots : (la commande est sur la même ligne) :

```
hadoop jar fsb_wcount.jar com.fsb.mapreduce.WCount input/poeme.txt /results
```

- La commande demandera à Hadoop d'exécuter le programme Hadoop de la classe Driver `com.fsb.mapreduce.WCount` du fichier `fsb_wcount.jar`, en lui indiquant qu'il doit lire son texte dans le fichier « `input/poeme.txt` » sur HDFS, et stocker les résultats dans le répertoire « `results` » sur HDFS.

Programmation Hadoop

Remarques :

- Hadoop stocke les résultats dans une série de fichiers part-r-XXXX, où XXXX est un compteur incrémental. L'idée est ici de stocker de grandes quantités de résultats dans de nombreux fichiers différents, qui seront en conséquences distribués sur tout le cluster HDFS. Le nom du fichier est paramétrable dans la configuration Hadoop.
- On a un fichier « part-r » par opération REDUCE exécutée. Le « r » au sein du nom signifie « Reduce ». On peut également demander à Hadoop d'effectuer uniquement les opérations MAP (par exemple pour du debug), auquel cas on aura une série de fichiers « part-m ».
- Un fichier _SUCCESS (vide) est également créé dans le répertoire des résultats en cas de succès. Cela permet de contrôler que tout s'est bien passé rapidement (avec un simple `hadoop fs -ls` sur HDFS).

Autre langages : Streaming (1/10)

- Au delà de Java, Hadoop permet également l'exécution d'un programme Hadoop écrit dans d'autres langages, par exemple en C, en Python ou encore en bash (shell scripting).
- Pour ce faire, un outil est distribué avec Hadoop: streaming. Il s'agit d'un .jar qui est capable de prendre en argument des programmes ou scripts définissant les tâches MAP et REDUCE, ainsi que les fichiers d'entrée et le répertoire de sortie HDFS, et d'exécuter ainsi la tâche spécifiée sur le cluster.
- Ce .jar est disponible dans le répertoire d'installation Hadoop et porte le nom `hadoop-streaming-VERSION.jar`, où **VERSION** est la version de Hadoop concernée.
- Il s'agit en réalité d'un programme Hadoop Java « classique », mais qui appelle les tâches MAP et REDUCE par le biais du système.

•

Autre langages : Streaming - Map (2/10)

Lorsqu'on développe un programme MAP Hadoop dans un autre langage pour son utilisation avec l'outil streaming, les données d'entrée doivent être lues sur l'entrée standard (stdin) et les données de sorties doivent être envoyées sur la sortie standard (stdout).

- En entrée du script ou programme MAP, on aura une série de lignes: nos données d'entrée (par exemple dans le cas du compteur d'occurrences de mots, des lignes de notre texte).
- En sortie du script ou programme MAP, on doit écrire sur stdout notre série de couples (clef;valeur) au format:

CLEF[**TABULATION**]**VALEUR**

... avec une ligne distincte pour chaque (clef;valeur).

Autre langages : Streaming - Reduce (3/10)

Lorsqu'on développe un programme REDUCE Hadoop dans un autre langage pour son utilisation avec l'outil streaming, les données d'entrée et de sortie doivent être lues/écrites sur stdin et stdout (respectivement).

- En entrée du script ou programme REDUCE, on aura une série de lignes: des couples (clef;valeur) au format: **CLEF** **[TABULATION]** **VALEUR**

Les couples seront triés par clef distincte, et la clef répétée à chaque fois. Par ailleurs, on est susceptible d'avoir des clefs différentes au sein d'une seule et même exécution du programme reduce !

- En sortie du script ou programme REDUCE, on doit écrire des couples (clef;valeur), toujours au format **CLEF** **[TABULATION]** **VALEUR**

Remarque: d'ordinaire, on écrira évidemment un seul **couple (clef;valeur)** par clef distincte

Autre langages : Streaming – Ex : Python (4/10)

Occurrences de mots version Python – opération MAP:

```
import sys

# Pour chaque ligne d'entrée.
for line in sys.stdin:
    # Supprimer les espaces autour de la ligne.
    line=line.strip()
    # Pour chaque mot de la ligne.
    words=line.split()
    for word in words:
        # Renvoyer couple clef;valeur: le mot comme clef, l'entier "1" comme
        # valeur.
        # On renvoie chaque couple sur une ligne, avec une tabulation entre
        # la clef et la valeur.
        print "%s\t%d" % (word, 1)
```

Autre langages : Streaming – Ex : Python (5/10)

Occurrences de mots version Python – opération Reduce:

```
import sys

total=0; # Notre total pour le mot courant.
# Contient le dernier mot rencontré.
lastword=None

# Pour chaque ligne d'entrée.
for line in sys.stdin:
    # Supprimer les espaces autour de la ligne.
    line=line.strip()

    # Récupérer la clef et la valeur, convertir la valeur en int.
    word, count=line.split('\t', 1)
    count=int(count)
```

Autre langages : Streaming – Ex : Python (6/10)

```
# On change de mot (test nécessaire parce qu'on est susceptible d'avoir
# en entrée plusieurs clefs distinctes différentes pour une seule et
# même exécution du programme - Hadoop triera les couples par clef
# distincte).
if word!=lastword and lastword!=None:
    print "%s\t%d occurrences" % (lastword, total)
    total=0;
lastword=word

total=total+count # Ajouter la valeur au total

# Ecrire le dernier couple (clef;valeur).
print "%s\t%d occurrences" % (lastword, total)
```

Autre langages : Streaming – Ex : Batch (7/10)

Occurrences de mots version Batch – opération MAP

```
# Pour chaque ligne d'entrée.
while read line; do
    # Supprimer les espaces autour de la ligne.
    line=$(echo "$line"|sed 's/^\s*\(.+\)\s*$/\1/')
    # Pour chaque mot de la ligne.
    for word in $line; do
        # Renvoyer couple clef;valeur: le mot comme clef, l'entier "1" comme
        # valeur.
        # On renvoie chaque couple sur une ligne, avec une tabulation entre
        # la clef et la valeur.
        echo -e $word"\t"1
    done
done
```

Autre langages : Streaming – Ex : Batch (8/10)

Occurrences de mots version Batch – opération Reduce

```
lastword=""
total=0

# Pour chaque ligne d'entrée.
while read line; do
    # Supprimer les espaces autour de la ligne.
    line=$(echo "$line"|sed 's/^\s*\(.\.+\)\s*$/\1/')

    # Recuperer mot et occurrence (IE, clef et valeur)
    word=$(echo "$line"|awk -F "\t" '{print $1}');
    nb=$(echo "$line"|awk -F "\t" '{print $2}');
```

Autre langages : Streaming – Ex : Batch (9/10)

```
# On change de mot (test nécessaire parce qu'on est susceptible d'avoir
# en entrée plusieurs clefs distinctes différentes pour une seule et
# même exécution du programme - Hadoop triera les couples par clef
# distincte).
if [ "$word" != "$lastword" ] && [ "$lastword" != "" ]; then
    echo -e $lastword"\t"$total" occurrences."
    total=0;
fi
lastword="$word"

total=$(( $total + $nb )) # Ajouter la valeur au total.
done

# Ecrire le dernier couple (clef;valeur).
echo -e $lastword"\t"$total" occurrences."
```


Autre langages : Streaming – Exécution (10/10)

Streaming est un programme Hadoop standard, on l'exécutera donc avec la même commande `hadoop jar` qu'un programme Hadoop Java habituel. C'est dans ses options de ligne de commande qu'on va indiquer les scripts ou programmes Hadoop map et reduce à utiliser. Syntaxe:

```
hadoop jar hadoop-streaming-X.Y.Z.jar -input [HDFS INPUT FILES] \  
                                         -output [HDFS OUTPUT FILES] \  
                                         -mapper [MAP PROGRAM] \  
                                         -reducer [REDUCE PROGRAM]
```

Par exemple:

```
hadoop jar hadoop-streaming-X.Y.Z.jar -input /poeme.txt \  
                                         -output /results -mapper ./map.py -reducer ./reduce.py
```

Après quoi la tâche Hadoop s'exécutera exactement comme une tâche standard.

Sources

- Cours :
 - Ce chapitre est pris essentiellement du cours :
 - Hadoop / Big Data - MBDS - Benjamin Renaut (avec quelques modifications).
 - Utilisation Hadoop – Amal Abid (avec quelques modifications)